

Customer Description

We want an **open, RTOS-based smart home and IoT automation platform** that serves as an alternative to ESPHome, but is built around **μT-Kernel (micro T-Kernel)** rather than the Arduino framework or bare-metal event loops.

The platform should enable users to rapidly develop, configure, and deploy smart devices using microcontrollers such as the ESP32 family. Users should be able to integrate common sensors, actuators, and communication interfaces without needing to write large amounts of low-level firmware. Instead, the platform should provide reusable software components, standardized device abstractions, and a simplified configuration-driven workflow.

The system should support a wide range of IoT devices, including:

- Environmental sensors (temperature, humidity, light, air quality, etc.)
- Switches and relays
- Smart lighting devices
- Displays and user interfaces
- Motor and actuator control
- Wireless sensor nodes
- Home automation gateways

Unlike traditional hobbyist frameworks, the platform should fully leverage the capabilities of a real-time operating system. Device functions should execute as independent tasks with appropriate prioritization, scheduling, synchronization, and fault isolation. This would allow complex applications consisting of multiple sensors, communication protocols, and control algorithms to operate reliably and predictably.

The platform should provide a library of reusable middleware components and drivers that can be composed into larger applications. Users should be able to combine communication stacks, sensor drivers, automation logic, cloud connectivity, and local interfaces through a common architecture rather than building each application from scratch.

The system should support popular IoT communication technologies, including:

- Wi-Fi
- Bluetooth Low Energy (BLE)
- MQTT
- HTTP/REST

- Local automation protocols
- Serial and industrial interfaces

Configuration, deployment, and monitoring should be designed to be simple enough for hobbyists and makers while remaining sufficiently robust for industrial and commercial applications.

The platform should also expose RTOS concepts that are normally hidden in frameworks such as ESPHome. Users should be able to observe, configure, and manage tasks, inter-task communication, timing behaviour, resource utilization, and system health. This allows the platform to serve not only as a practical automation framework but also as an educational platform for embedded and real-time systems.

The completed solution should enable users to build smart home and IoT systems using a configuration-centric workflow similar to ESPHome while benefiting from the determinism, scalability, modularity, and reliability offered by μ T-Kernel.

Motivation

Current IoT frameworks such as ESPHome make it easy to develop and deploy smart devices but are primarily focused on simplicity and rapid development.

As IoT systems become more complex, there is increasing demand for:

- Predictable real-time behaviour
- Better task isolation
- Structured concurrency
- Improved scalability
- Stronger software architecture practices
- Educational exposure to RTOS concepts

This project seeks to bridge the gap between ease of use and real-time systems engineering by creating a modern IoT automation framework that combines the user experience of ESPHome with the architecture and reliability of μ T-Kernel.

Short Product Vision Statement

Develop a **μ T-Home**, an RTOS-native alternative to ESPHome that enables rapid creation of smart home and IoT devices through reusable components, configuration-driven development, and the reliability of μ T-Kernel.

Requirements Students Can Infer

Area	Questions students should resolve
Device Model	How should sensors, actuators, and services be represented?
Configuration System	Should devices be configured through YAML, JSON, GUI tools, or another approach?
RTOS Architecture	How are tasks created, prioritized, and managed?
Component Framework	How can reusable drivers and middleware be integrated consistently?
Device Discovery	How can users discover and manage deployed devices?
Communications	Which protocols should be supported and how are they abstracted?
Automation Engine	How should rules, triggers, and actions be represented?
Resource Management	How can memory, queues, and timers be monitored and controlled?
Observability	How can task execution, CPU usage, and system health be visualized?
Security	How should authentication, encryption, and firmware updates be handled?
Extensibility	How easily can third-party components be added?
Deployment	How should users build, flash, and update devices?
Reliability	How should task failures, communication errors, and resource exhaustion be handled?
Scalability	How can the architecture support increasingly complex IoT applications?